

Semana 5 - Aspectos Práticos do *Deep Learning*

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

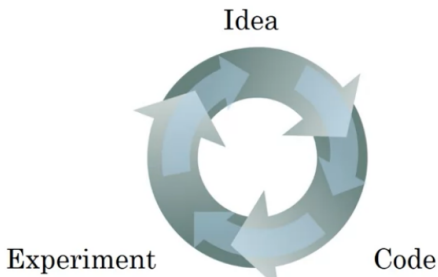
1 Treino, Desenvolvimento e Teste

2 Viés/Variância

3 Regularização

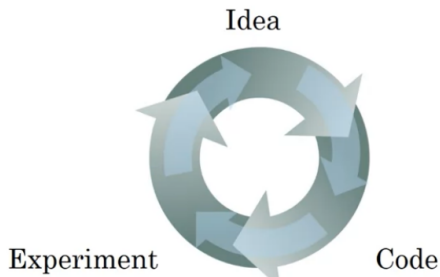
4 Configuração da Otimização

- Processo cíclico e iterativo



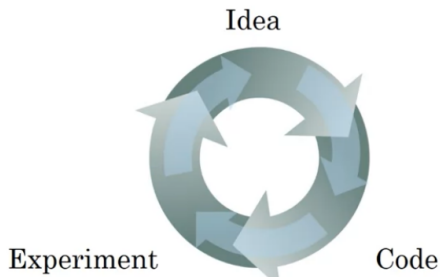
- Decisões importantes, como o número de camadas e de unidades escondidas, a taxa de aprendizado, as funções de ativação, etc.
- Tomadas em vários ciclos de desenvolvimento: quanto mais eficiente esse processo, melhor
- Muitas áreas de aplicação, prática de uma área pode não valer na outra
- Depende também dos dados e recursos computacionais

- Processo cíclico e iterativo



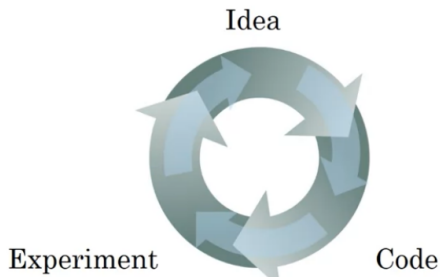
- Decisões importantes, como o número de camadas e de unidades escondidas, a taxa de aprendizado, as funções de ativação, etc.
- Tomadas em vários ciclos de desenvolvimento: quanto mais eficiente esse processo, melhor
- Muitas áreas de aplicação, prática de uma área pode não valer na outra
- Depende também dos dados e recursos computacionais

- Processo cíclico e iterativo



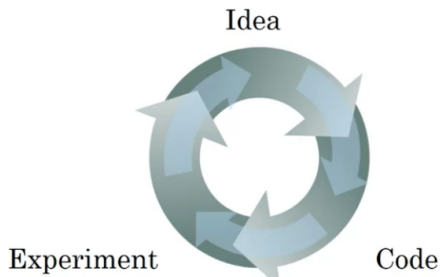
- Decisões importantes, como o número de camadas e de unidades escondidas, a taxa de aprendizado, as funções de ativação, etc.
- Tomadas em vários ciclos de desenvolvimento: quanto mais eficiente esse processo, melhor
- Muitas áreas de aplicação, prática de uma área pode não valer na outra
- Depende também dos dados e recursos computacionais

- Processo cíclico e iterativo



- Decisões importantes, como o número de camadas e de unidades escondidas, a taxa de aprendizado, as funções de ativação, etc.
- Tomadas em vários ciclos de desenvolvimento: quanto mais eficiente esse processo, melhor
- Muitas áreas de aplicação, prática de uma área pode não valer na outra
- Depende também dos dados e recursos computacionais

- Processo cíclico e iterativo



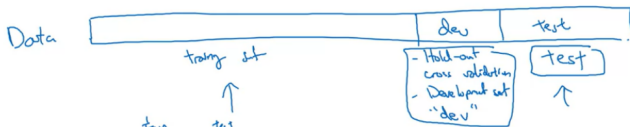
- Decisões importantes, como o número de camadas e de unidades escondidas, a taxa de aprendizado, as funções de ativação, etc.
- Tomadas em vários ciclos de desenvolvimento: quanto mais eficiente esse processo, melhor
- Muitas áreas de aplicação, prática de uma área pode não valer na outra
- Depende também dos dados e recursos computacionais

Treino, Teste e Desenvolvimento



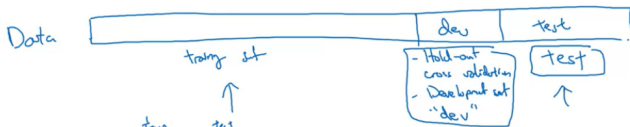
- Conjunto “dev” usado para escolher o melhor modelo treinado
- Avaliado ao final no teste
- 70% treino/30% teste (ou 60%/20%/20%) era padrão: perfeito para 100, 1000 ou 10000 exemplos
- No *big data* (1 milhão+ exemplos), porcentagem de dado para dev e teste pode ser muito menor
- EX.: 1 milhão de exemplos, 10000 para dev e 10000 para teste (98%/1%/1%) (98%/1%/1%); até 99%/0.5%/0.5% ou 99%/0.4%/0.1% poderia funcionar!

Treino, Teste e Desenvolvimento



- Conjunto “dev” usado para escolher o melhor modelo treinado
- Avaliado ao final no teste
- 70% treino/30% teste (ou 60%/20%/20%) era padrão: perfeito para 100, 1000 ou 10000 exemplos
- No *big data* (1 milhão+ exemplos), porcentagem de dado para dev e teste pode ser muito menor
- EX.: 1 milhão de exemplos, 10000 para dev e 10000 para teste (98%/1%/1%) (98%/1%/1%); até 99%/0.5%/0.5% ou 99%/0.4%/0.1% poderia funcionar!

Treino, Teste e Desenvolvimento



- Conjunto “dev” usado para escolher o melhor modelo treinado
- Avaliado ao final no teste
- 70% treino/30% teste (ou 60%/20%/20%) era padrão: perfeito para 100, 1000 ou 10000 exemplos
- No *big data* (1 milhão+ exemplos), porcentagem de dado para dev e teste pode ser muito menor
- EX.: 1 milhão de exemplos, 10000 para dev e 10000 para teste (98%/1%/1%) (98%/1%/1%); até 99%/0.5%/0.5% ou 99%/0.4%/0.1% poderia funcionar!

Treino, Teste e Desenvolvimento



- Conjunto “dev” usado para escolher o melhor modelo treinado
- Avaliado ao final no teste
- 70% treino/30% teste (ou 60%/20%/20%) era padrão: perfeito para 100, 1000 ou 10000 exemplos
- No *big data* (1 milhão+ exemplos), porcentagem de dado para dev e teste pode ser muito menor
- EX.: 1 milhão de exemplos, 10000 para dev e 10000 para teste (98%/1%/1%) (98%/1%/1%); até 99%/0.5%/0.5% ou 99%/0.4%/0.1% poderia funcionar!

Treino, Teste e Desenvolvimento



- Conjunto “dev” usado para escolher o melhor modelo treinado
- Avaliado ao final no teste
- 70% treino/30% teste (ou 60%/20%/20%) era padrão: perfeito para 100, 1000 ou 10000 exemplos
- No *big data* (1 milhão+ exemplos), porcentagem de dado para dev e teste pode ser muito menor
- EX.: 1 milhão de exemplos, 10000 para dev e 10000 para teste (98%/1%/1%) (98%/1%/1%); até 99%/0.5%/0.5% ou 99%/0.4%/0.1% poderia funcionar!

- Comum no *deep learning* uma certa discrepância entre a distribuição do treino e do teste
- EX.: app de reconhecimento de gatos treinado com imagens da web e com dev e teste fornecidos pelos usuários
- Certifique-se de que os conjuntos de desenvolvimento e teste venham da **mesma distribuição**
- Não ter um conjunto de teste (apenas dev) pode ser OK; é comum aqui que se chame o *dev set* de *test set*

- Comum no *deep learning* uma certa discrepância entre a distribuição do treino e do teste
- EX.: app de reconhecimento de gatos treinado com imagens da web e com dev e teste fornecidos pelos usuários
- Certifique-se de que os conjuntos de desenvolvimento e teste venham da **mesma distribuição**
- Não ter um conjunto de teste (apenas dev) pode ser OK; é comum aqui que se chame o *dev set* de *test set*

- Comum no *deep learning* uma certa discrepância entre a distribuição do treino e do teste
- EX.: app de reconhecimento de gatos treinado com imagens da web e com dev e teste fornecidos pelos usuários
- Certifique-se de que os conjuntos de desenvolvimento e teste venham da **mesma distribuição**
- Não ter um conjunto de teste (apenas dev) pode ser OK; é comum aqui que se chame o *dev set* de *test set*

- Comum no *deep learning* uma certa discrepância entre a distribuição do treino e do teste
- EX.: app de reconhecimento de gatos treinado com imagens da web e com dev e teste fornecidos pelos usuários
- Certifique-se de que os conjuntos de desenvolvimento e teste venham da **mesma distribuição**
- Não ter um conjunto de teste (apenas dev) pode ser OK; é comum aqui que se chame o *dev set* de *test set*

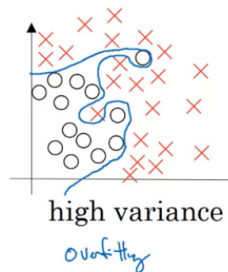
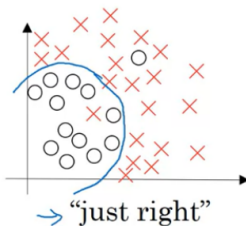
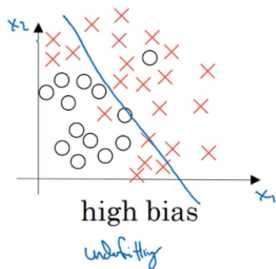
Outline

- 1 Treino, Desenvolvimento e Teste
- 2 **Viés/Variância**
- 3 Regularização
- 4 Configuração da Otimização

- Conceito fácil de entender em princípio, mas difícil de dominar!
- Não é algo tão crítico nem tão discutido em *deep learning*

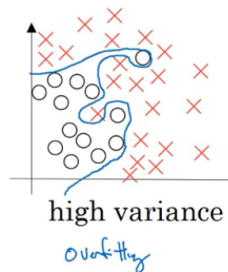
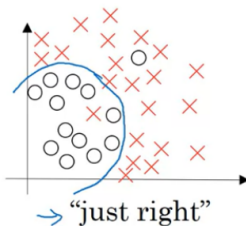
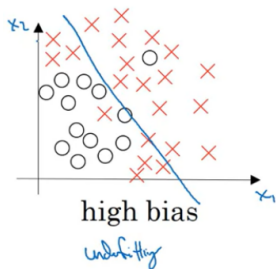
- Diagnóstico pelo erro no treino e no dev

- Conceito fácil de entender em princípio, mas difícil de dominar!
- Não é algo tão crítico nem tão discutido em *deep learning*



- Diagnóstico pelo erro no treino e no dev

- Conceito fácil de entender em princípio, mas difícil de dominar!
- Não é algo tão crítico nem tão discutido em *deep learning*



- Diagnóstico pelo erro no treino e no dev



Tipo de erro	Variância alta	Viés alto	Viés alto, variância alta	viés baixo, variância baixa
Treino	1%	15%	15%	0.5%
Dev	11%	16%	30%	1%

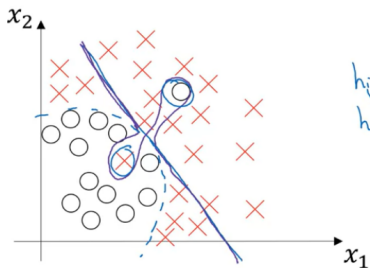
Note que assumimos erro ótimo (erro de Bayes) $\sim 0\%$

Alto viés e alta variância é o pior dos dois mundos:

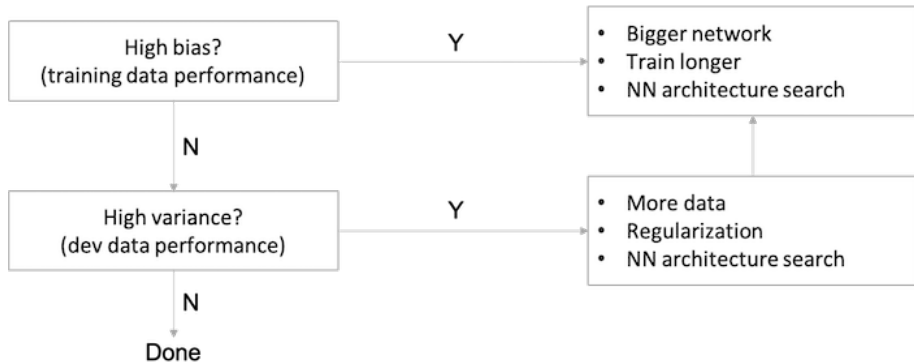
Tipo de erro	Variância alta	Viés alto	Viés alto, variância alta	viés baixo, variância baixa
Treino	1%	15%	15%	0.5%
Dev	11%	16%	30%	1%

Note que assumimos erro ótimo (erro de Bayes) $\sim 0\%$

Alto viés e alta variância é o pior dos dois mundos:



Receita Básica



Receita Básica

- Cada problema tem solução diferente: EX.: aumentar os dados não diminui viés
- Dilema viés/variância não é mais crítico no *deep learning*
- Podemos treinar redes muito maiores e adquirir mais e mais dados
- Assim, usando um modelo regularizado adequadamente, pode-se reduzir viés e variância ao mesmo tempo
- Mais uma razão do sucesso de *deep learning*

Receita Básica

- Cada problema tem solução diferente: EX.: aumentar os dados não diminui viés
- Dilema viés/variância não é mais crítico no *deep learning*
- Podemos treinar redes muito maiores e adquirir mais e mais dados
- Assim, usando um modelo regularizado adequadamente, pode-se reduzir viés e variância ao mesmo tempo
- Mais uma razão do sucesso de *deep learning*

Receita Básica

- Cada problema tem solução diferente: EX.: aumentar os dados não diminui viés
- Dilema viés/variância não é mais crítico no *deep learning*
- Podemos treinar redes muito maiores e adquirir mais e mais dados
- Assim, usando um modelo regularizado adequadamente, pode-se reduzir viés e variância ao mesmo tempo
- Mais uma razão do sucesso de *deep learning*

Receita Básica

- Cada problema tem solução diferente: EX.: aumentar os dados não diminui viés
- Dilema viés/variância não é mais crítico no *deep learning*
- Podemos treinar redes muito maiores e adquirir mais e mais dados
- Assim, usando um modelo regularizado adequadamente, pode-se reduzir viés e variância ao mesmo tempo
- Mais uma razão do sucesso de *deep learning*

Receita Básica

- Cada problema tem solução diferente: EX.: aumentar os dados não diminui viés
- Dilema viés/variância não é mais crítico no *deep learning*
- Podemos treinar redes muito maiores e adquirir mais e mais dados
- Assim, usando um modelo regularizado adequadamente, pode-se reduzir viés e variância ao mesmo tempo
- Mais uma razão do sucesso de *deep learning*

Outline

- 1 Treino, Desenvolvimento e Teste
- 2 Viés/Variância
- 3 Regularização**
- 4 Configuração da Otimização

- Uma forma segura de reduzir variância e prevenir *overfitting* é aumentando dados
- Outra forma é usando **regularização**
- Regressão logística:

NOTA: b é apenas um entre muitos outros parâmetros W , por isso não precisa entrar

- Uma forma segura de reduzir variância e prevenir *overfitting* é aumentando dados
- Outra forma é usando **regularização**
- Regressão logística:

NOTA: b é apenas um entre muitos outros parâmetros W , por isso não precisa entrar

- Uma forma segura de reduzir variância e prevenir *overfitting* é aumentando dados
- Outra forma é usando **regularização**
- Regressão logística:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|\omega\|_2^2$$

NOTA: b é apenas um entre muitos outros parâmetros W , por isso não precisa entrar

Outros Tipos de Regularização

regularização	fórmula	descrição
regularização L2	$\ w\ _2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$	tipo mais comum
regularização L1	$\ w\ _1 = \sum_{j=1}^{n_x} w_j $	vetor w esparso

- Norma L1 é mais usada quando se deseja comprimir o modelo
- NOTA 1: Lambda é chamado de **parâmetro de regularização** e é mais um hiperparâmetro a ser definido usando o dev set
- NOTA 2: Lambda é uma palavra reservada em Python!

Outros Tipos de Regularização

regularização	fórmula	descrição
regularização L2	$\ w\ _2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$	tipo mais comum
regularização L1	$\ w\ _1 = \sum_{j=1}^{n_x} w_j $	vetor w esparso

- Norma L1 é mais usada quando se deseja comprimir o modelo
- NOTA 1: Lambda é chamado de **parâmetro de regularização** e é mais um hiperparâmetro a ser definido usando o dev set
- NOTA 2: Lambda é uma palavra reservada em Python!

Outros Tipos de Regularização

regularização	fórmula	descrição
regularização L2	$\ w\ _2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$	tipo mais comum
regularização L1	$\ w\ _1 = \sum_{j=1}^{n_x} w_j $	vetor w esparso

- Norma L1 é mais usada quando se deseja comprimir o modelo
- NOTA 1: Lambda é chamado de **parâmetro de regularização** e é mais um hiperparâmetro a ser definido usando o dev set
- NOTA 2: Lambda é uma palavra reservada em Python!

Outros Tipos de Regularização

regularização	fórmula	descrição
regularização L2	$\ w\ _2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$	tipo mais comum
regularização L1	$\ w\ _1 = \sum_{j=1}^{n_x} w_j $	vetor w esparso

- Norma L1 é mais usada quando se deseja comprimir o modelo
- NOTA 1: Lambda é chamado de **parâmetro de regularização** e é mais um hiperparâmetro a ser definido usando o dev set
- NOTA 2: Lambda é uma palavra reservada em Python!

Regularização na Rede Neural

Custo:

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|\omega^{[l]}\|_F^2$$

A norma da matriz W lembra L2, mas é chamada de **norma de Frobenius**:

$$\|\omega^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (\omega_{i,j}^{[l]})^2$$

Regularização do Gradiente

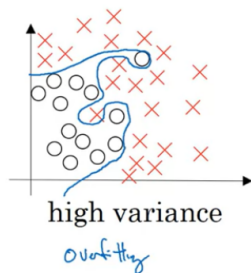
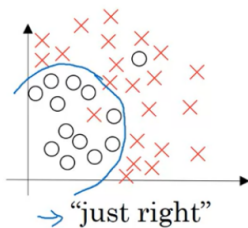
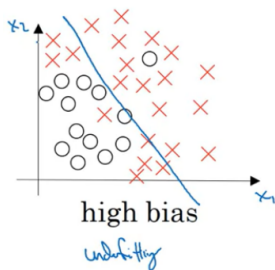
Temos no *backpropagation*:

$$d\omega^{[l]} = (\text{backprop}) + \frac{\lambda}{m}\omega^{[l]}$$

Usa-se o termo *weight decay* porque o valor do próprio peso reduzido por um fator que inclui λ é adicionado na atualização do gradiente descendente:

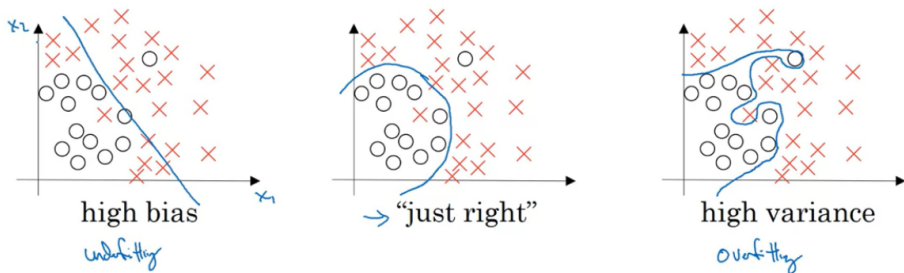
$$\begin{aligned}\omega^{[l]} &= \omega^{[l]} - \alpha d\omega^{[l]} \\ &= \omega^{[l]} - \alpha[(\text{backprop}) + \frac{\lambda}{m}\omega^{[l]}] \\ &= (1 - \frac{\alpha\lambda}{m})\omega^{[l]} - \alpha(\text{backprop})\end{aligned}$$

Por que a Regularização Reduz o *Overfitting*?

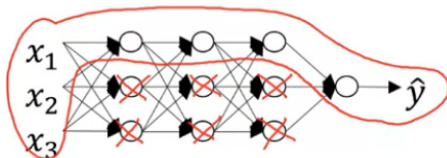


λ ser muito grande leva os pesos W para zero; no limite, temos a regressão logística

Por que a Regularização Reduz o *Overfitting*?



λ ser muito grande leva os pesos W para zero; no limite, temos a regressão logística

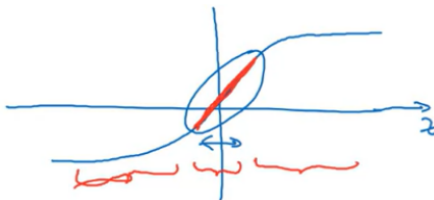


Outra Intuição

- λ grande $\rightarrow W \approx 0 \rightarrow z \approx 0$
- Funções de ativação como $g(z) = \tanh(z)$ são quase lineares para $z \approx 0$
- Composição de camadas lineares é uma função linear

Outra Intuição

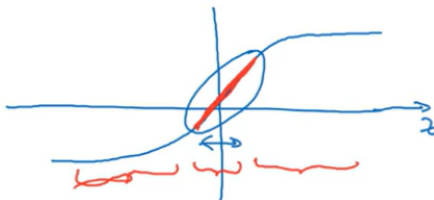
- λ grande $\rightarrow W \approx 0 \rightarrow z \approx 0$
- Funções de ativação como $g(z) = \tanh(z)$ são quase lineares para $z \approx 0$



- Composição de camadas lineares é uma função linear

Outra Intuição

- λ grande $\rightarrow W \approx 0 \rightarrow z \approx 0$
- Funções de ativação como $g(z) = \tanh(z)$ são quase lineares para $z \approx 0$

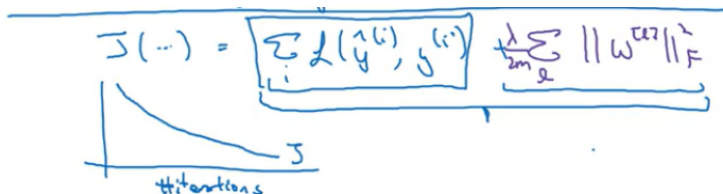


- Composição de camadas lineares é uma função linear

Nota

ATENÇÃO: No gráfico do custo J por iteração, não se esqueça de incluir o termo regularizador

Do contrário, poderá não ver uma diminuição em cada época



Dropout

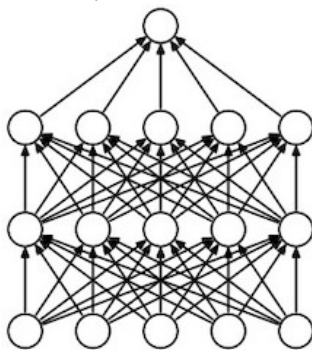
- Outra técnica poderosa de regularização
- Remove nós (entradas e saídas setadas para zero) de uma camada com determinada probabilidade
- Como se em cada iteração tivesse uma rede menor (efeito regularizador)

Dropout

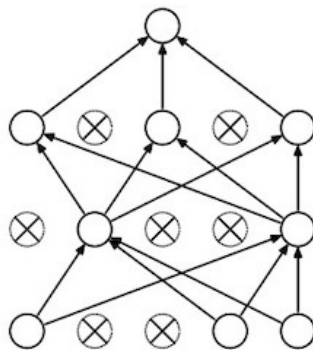
- Outra técnica poderosa de regularização
- Remove nós (entradas e saídas setadas para zero) de uma camada com determinada probabilidade
- Como se em cada iteração tivesse uma rede menor (efeito regularizador)

Dropout

- Outra técnica poderosa de regularização
- Remove nós (entradas e saídas setadas para zero) de uma camada com determinada probabilidade
- Como se em cada iteração tivesse uma rede menor (efeito regularizador)



(a) Standard Neural Net



(b) After applying dropout.

Implementação

keep_prob é a probabilidade de manter o nó. EX. na camada $l = 3$:

```
d3 = np.random.rand(a3.shape[0], a3.shape[1]) < keep_prob
a3 = np.multiply(a3, d3)
a3 /= keep_prob
```

NOTA: *d3* é uma matriz Booleana, mas o Python entende como de zeros (False) e uns (True).

- Última linha faz o *dropout* invertido
- Note que o valor esperado de *a3* é multiplicado por *keep_prob*
- O mesmo ocorre com *z*, já que $z^{[4]} = W^{[4]}a^{[3]} + b^{[4]}$
- O *dropout* invertido corrige essa distorção, facilitando na fase de teste

Implementação

keep_prob é a probabilidade de manter o nó. EX. na camada $l = 3$:

```
d3 = np.random.rand(a3.shape[0], a3.shape[1]) < keep_prob
a3 = np.multiply(a3, d3)
a3 /= keep_prob
```

NOTA: *d3* é uma matriz Booleana, mas o Python entende como de zeros (False) e uns (True).

- Última linha faz o *dropout* invertido
- Note que o valor esperado de *a3* é multiplicado por *keep_prob*
- O mesmo ocorre com *z*, já que $z^{[4]} = W^{[4]}a^{[3]} + b^{[4]}$
- O *dropout* invertido corrige essa distorção, facilitando na fase de teste

Implementação

keep_prob é a probabilidade de manter o nó. EX. na camada $l = 3$:

```
d3 = np.random.rand(a3.shape[0], a3.shape[1]) < keep_prob
a3 = np.multiply(a3, d3)
a3 /= keep_prob
```

NOTA: *d3* é uma matriz Booleana, mas o Python entende como de zeros (False) e uns (True).

- Última linha faz o *dropout* invertido
- Note que o valor esperado de *a3* é multiplicado por *keep_prob*
- O mesmo ocorre com *z*, já que $z^{[4]} = W^{[4]}a^{[3]} + b^{[4]}$
- O *dropout* invertido corrige essa distorção, facilitando na fase de teste

Observações

- Unidades removidas podem ser diferentes em cada exemplo de treinamento e cada iteração do gradiente
- Não se faz *dropout* no teste: não queremos uma predição aleatória

Observações

- Unidades removidas podem ser diferentes em cada exemplo de treinamento e cada iteração do gradiente
- Não se faz *dropout* no teste: não queremos uma predição aleatória

Por que Funciona

- Vimos que equivale a treinar uma rede menor, previne *overfitting*
- Força cada unidade não depender de um único *feature*
- Espalha o valor dos pesos $\Rightarrow$ reduz sua magnitude
- Similar à regularização L2

Por que Funciona

- Vimos que equivale a treinar uma rede menor, previne *overfitting*
- Força cada unidade não depender de um único *feature*
- Espalha o valor dos pesos $\Rightarrow$ reduz sua magnitude
- Similar à regularização L2

Por que Funciona

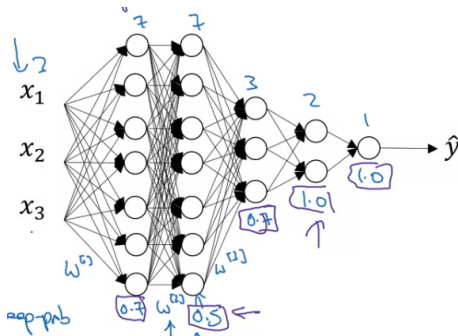
- Vimos que equivale a treinar uma rede menor, previne *overfitting*
- Força cada unidade não depender de um único *feature*
- Espalha o valor dos pesos $\Rightarrow$ reduz sua magnitude
- Similar à regularização L2

Por que Funciona

- Vimos que equivale a treinar uma rede menor, previne *overfitting*
- Força cada unidade não depender de um único *feature*
- Espalha o valor dos pesos $\Rightarrow$ reduz sua magnitude
- Similar à regularização L2

Observações

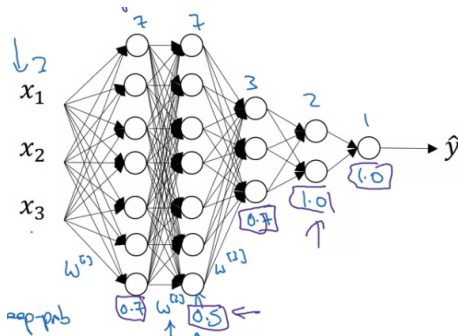
- *keep_prob* não precisa ser igual em todas as camadas
- *keep_prob* menor em camadas com mais entradas e saídas (mais propensas a *overfitting*)



- Problema: mais hiperparâmetros!

Observações

- *keep_prob* não precisa ser igual em todas as camadas
- *keep_prob* menor em camadas com mais entradas e saídas (mais propensas a *overfitting*)



- Problema: mais hiperparâmetros!

Observações

- Não se costuma usar *dropout* na entrada
- Mais popular em visão computacional; entradas muito grandes (pixels) sem exemplos suficientes para treinar (*overfitting*)
- A checagem se a curva de J diminui em cada iteração não é mais válida
- O que se pode é desligar o *dropout* para observar J

Observações

- Não se costuma usar *dropout* na entrada
- Mais popular em visão computacional; entradas muito grandes (pixels) sem exemplos suficientes para treinar (*overfitting*)
- A checagem se a curva de J diminui em cada iteração não é mais válida
- O que se pode é desligar o *dropout* para observar J

Observações

- Não se costuma usar *dropout* na entrada
- Mais popular em visão computacional; entradas muito grandes (pixels) sem exemplos suficientes para treinar (*overfitting*)
- A checagem se a curva de J diminui em cada iteração não é mais válida
- O que se pode é desligar o *dropout* para observar J

Observações

- Não se costuma usar *dropout* na entrada
- Mais popular em visão computacional; entradas muito grandes (pixels) sem exemplos suficientes para treinar (*overfitting*)
- A checagem se a curva de J diminui em cada iteração não é mais válida
- O que se pode é desligar o *dropout* para observar J

Data Augmentation

- Aumentar o conjunto de treino previne *overfitting*, mas normalmente é custoso
- *Data augmentation* aumenta o treino “de graça”
- Espelhamento horizontal, rotação e zoom aleatório, distorções aleatórias, etc.

NOTA: A distorção no dígito acima está exagerada. Normalmente é algo bem mais sutil.

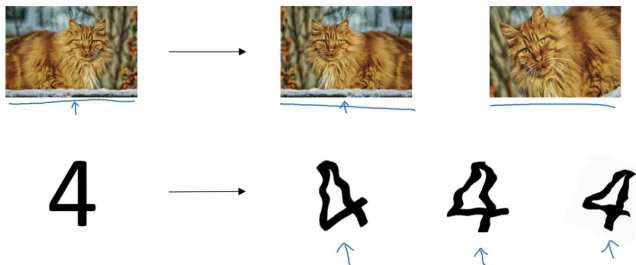
Data Augmentation

- Aumentar o conjunto de treino previne *overfitting*, mas normalmente é custoso
- *Data augmentation* aumenta o treino “de graça”
- Espelhamento horizontal, rotação e zoom aleatório, distorções aleatórias, etc.

NOTA: A distorção no dígito acima está exagerada. Normalmente é algo bem mais sutil.

Data Augmentation

- Aumentar o conjunto de treino previne *overfitting*, mas normalmente é custoso
- *Data augmentation* aumenta o treino “de graça”
- Espelhamento horizontal, rotação e zoom aleatório, distorções aleatórias, etc.



NOTA: A distorção no dígito acima está exagerada. Normalmente é algo bem mais sutil.

Early Stopping

- Observamos a curva do erro de treino (ou J) e do erro de dev simultaneamente
- Quando o erro de dev começa a aumentar, paramos de treinar
- Intuição: W é inicializado próximo de zero e vai aumentando no decorrer das épocas
- Parando no meio, pegamos w com valores ainda medianos

Early Stopping

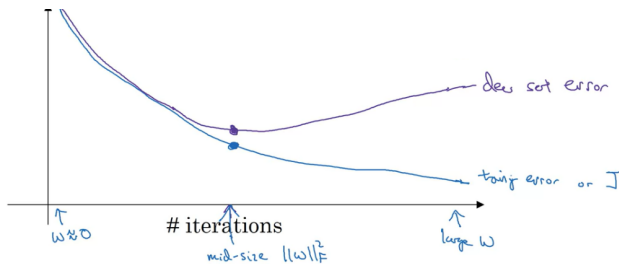
- Observamos a curva do erro de treino (ou J) e do erro de dev simultaneamente
- Quando o erro de dev começa a aumentar, paramos de treinar
- Intuição: W é inicializado próximo de zero e vai aumentando no decorrer das épocas
- Parando no meio, pegamos w com valores ainda medianos

Early Stopping

- Observamos a curva do erro de treino (ou J) e do erro de dev simultaneamente
- Quando o erro de dev começa a aumentar, paramos de treinar
- Intuição: W é inicializado próximo de zero e vai aumentando no decorrer das épocas
- Parando no meio, pegamos w com valores ainda medianos

Early Stopping

- Observamos a curva do erro de treino (ou J) e do erro de dev simultaneamente
- Quando o erro de dev começa a aumentar, paramos de treinar
- Intuição: W é inicializado próximo de zero e vai aumentando no decorrer das épocas
- Parando no meio, pegamos w com valores ainda medianos



Early Stopping

- PROBLEMA: Quebra o princípio da **ortogonalização**
- Processo 1: otimização da função de custo J - escolha do otimizador (gradiente descendente, momento, Adam, etc.), número de épocas, etc.
- Processo 2: evitar *overfitting* - regularização, adquirir mais dados, etc.
- Devem ser processos independentes
- *Early stopping* quebra isso
- Processo de decisão em cada etapa mais complexo

Early Stopping

- PROBLEMA: Quebra o princípio da **ortogonalização**
- Processo 1: otimização da função de custo J - escolha do otimizador (gradiente descendente, momento, Adam, etc.), número de épocas, etc.
- Processo 2: evitar *overfitting* - regularização, adquirir mais dados, etc.
- Devem ser processos independentes
- *Early stopping* quebra isso
- Processo de decisão em cada etapa mais complexo

Early Stopping

- PROBLEMA: Quebra o princípio da **ortogonalização**
- Processo 1: otimização da função de custo J - escolha do otimizador (gradiente descendente, momento, Adam, etc.), número de épocas, etc.
- Processo 2: evitar *overfitting* - regularização, adquirir mais dados, etc.
- Devem ser processos independentes
- *Early stopping* quebra isso
- Processo de decisão em cada etapa mais complexo

Early Stopping

- PROBLEMA: Quebra o princípio da **ortogonalização**
- Processo 1: otimização da função de custo J - escolha do otimizador (gradiente descendente, momento, Adam, etc.), número de épocas, etc.
- Processo 2: evitar *overfitting* - regularização, adquirir mais dados, etc.
- Devem ser processos independentes
- *Early stopping* quebra isso
- Processo de decisão em cada etapa mais complexo

Early Stopping

- PROBLEMA: Quebra o princípio da **ortogonalização**
- Processo 1: otimização da função de custo J - escolha do otimizador (gradiente descendente, momento, Adam, etc.), número de épocas, etc.
- Processo 2: evitar *overfitting* - regularização, adquirir mais dados, etc.
- Devem ser processos independentes
- *Early stopping* quebra isso
- Processo de decisão em cada etapa mais complexo

Early Stopping

- PROBLEMA: Quebra o princípio da **ortogonalização**
- Processo 1: otimização da função de custo J - escolha do otimizador (gradiente descendente, momento, Adam, etc.), número de épocas, etc.
- Processo 2: evitar *overfitting* - regularização, adquirir mais dados, etc.
- Devem ser processos independentes
- *Early stopping* quebra isso
- Processo de decisão em cada etapa mais complexo

Early Stopping

- VANTAGEM: não precisa de um hiperparâmetro λ a mais
- Se isso porém não for um grande problema, a regularização L2 é preferível

Early Stopping

- VANTAGEM: não precisa de um hiperparâmetro λ a mais
- Se isso porém não for um grande problema, a regularização L2 é preferível

Outline

- 1 Treino, Desenvolvimento e Teste
- 2 Viés/Variância
- 3 Regularização
- 4 Configuração da Otimização

Normalização das Entradas

- Ajuda a acelerar o treinamento
- Duas etapas: subtrair a média (deixar com média 0) e divisão pela variância (deixar com variância 1)

Subtrair média

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$x := x - \mu$$

Dividir pela variância

$$\sigma = \frac{1}{m} \sum_{i=1}^m x^{(i)} ** 2$$

$$x = x / \sigma$$

IMPORTANTE: Usar **sempre os mesmos** μ e σ do treino para normalizar o conjunto de teste.

Normalização das Entradas

- Ajuda a acelerar o treinamento
- Duas etapas: subtrair a média (deixar com média 0) e divisão pela variância (deixar com variância 1)

Subtrair média

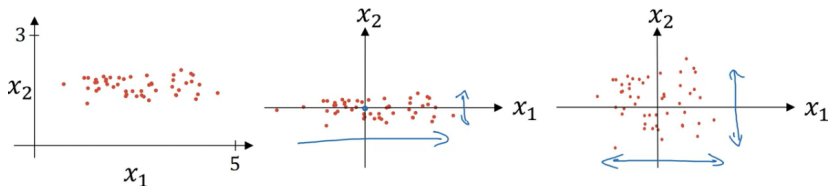
$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$x := x - \mu$$

Dividir pela variância

$$\sigma = \frac{1}{m} \sum_{i=1}^m x^{(i)} ** 2$$

$$x = x / \sigma$$



IMPORTANTE: Usar **sempre os mesmos** μ e σ do treino para normalizar o conjunto de teste.

Normalização das Entradas

- Ajuda a acelerar o treinamento
- Duas etapas: subtrair a média (deixar com média 0) e divisão pela variância (deixar com variância 1)

Subtrair média

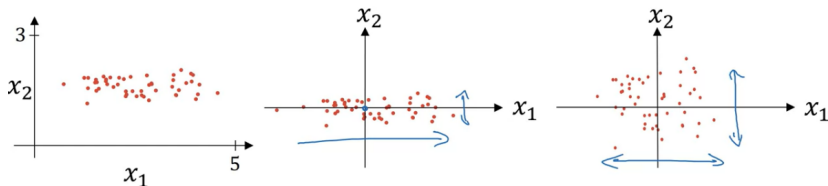
$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$x := x - \mu$$

Dividir pela variância

$$\sigma = \frac{1}{m} \sum_{i=1}^m x^{(i)} ** 2$$

$$x = x / \sigma$$



IMPORTANTE: Usar **sempre os mesmos** μ e σ do treino para normalizar o conjunto de teste.

Normalização das Entradas

- Ajuda a acelerar o treinamento
- Duas etapas: subtrair a média (deixar com média 0) e divisão pela variância (deixar com variância 1)

Subtrair média

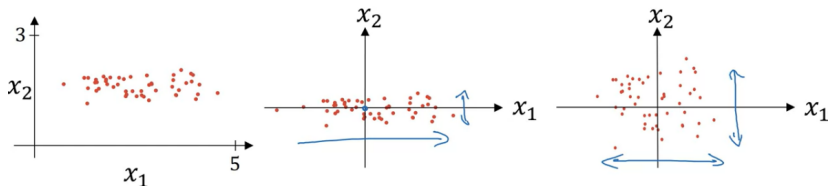
$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$x := x - \mu$$

Dividir pela variância

$$\sigma = \frac{1}{m} \sum_{i=1}^m x^{(i)} ** 2$$

$$x = x / \sigma$$



IMPORTANTE: Usar **sempre os mesmos** μ e σ do treino para normalizar o conjunto de teste.

Por que Normalizar?

- *Features* variando em intervalos muito diferentes geram uma superfície da função de custo mais alongada
- Trajetória do gradiente descendente em zigue-zague
- *Features* normalizados implicam e uma função de custo com formato mais circular
- Gradiente descendente vai diretamente para o mínimo
- Crítico de fato quando os intervalos são muito diferentes; diferenças pequenas não comprometem

Por que Normalizar?

- *Features* variando em intervalos muito diferentes geram uma superfície da função de custo mais alongada
- Trajetória do gradiente descendente em zigue-zague
- *Features* normalizados implicam e uma função de custo com formato mais circular
- Gradiente descendente vai diretamente para o mínimo
- Crítico de fato quando os intervalos são muito diferentes; diferenças pequenas não comprometem

Por que Normalizar?

- *Features* variando em intervalos muito diferentes geram uma superfície da função de custo mais alongada
- Trajetória do gradiente descendente em zigue-zague
- *Features* normalizados implicam e uma função de custo com formato mais circular
- Gradiente descendente vai diretamente para o mínimo
- Crítico de fato quando os intervalos são muito diferentes; diferenças pequenas não comprometem

Por que Normalizar?

- *Features* variando em intervalos muito diferentes geram uma superfície da função de custo mais alongada
- Trajetória do gradiente descendente em zigue-zague
- *Features* normalizados implicam e uma função de custo com formato mais circular
- Gradiente descendente vai diretamente para o mínimo
- Crítico de fato quando os intervalos são muito diferentes; diferenças pequenas não comprometem

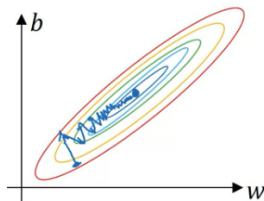
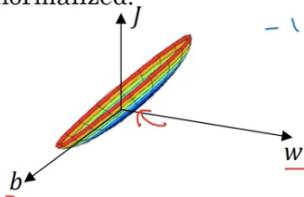
Por que Normalizar?

- *Features* variando em intervalos muito diferentes geram uma superfície da função de custo mais alongada
- Trajetória do gradiente descendente em zigue-zague
- *Features* normalizados implicam e uma função de custo com formato mais circular
- Gradiente descendente vai diretamente para o mínimo
- Crítico de fato quando os intervalos são muito diferentes; diferenças pequenas não comprometem

Por que Normalizar?

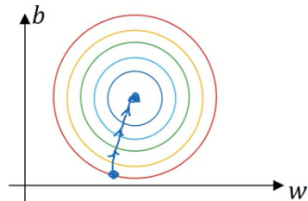
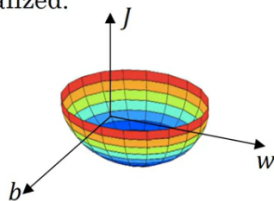
Unnormalized:

$w_1, x_1: 1 \dots 1000 \leftarrow$
 $w_2, x_2: 0 \dots 1 \leftarrow$
 $-1 \dots 1$



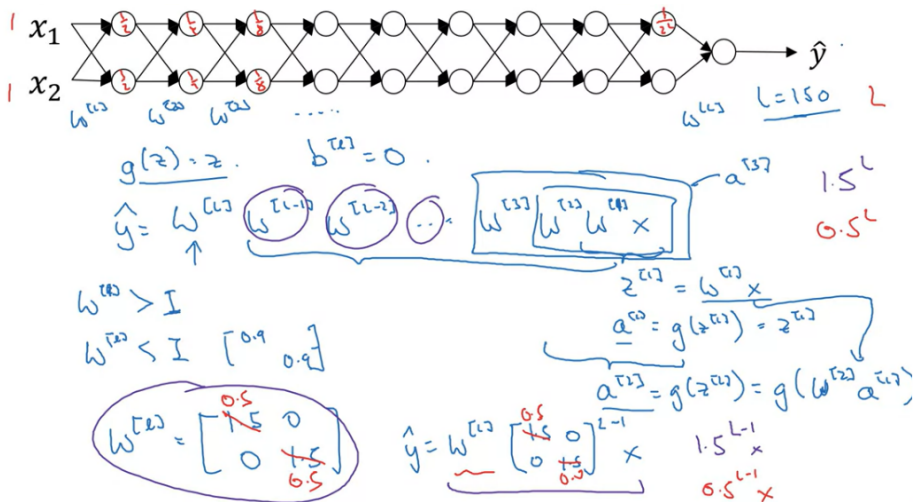
$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Normalized:



Explosão e Desaparecimento do Gradiente

Veja a rede profunda com 2 unidades por camada e sem *bias*:



Explosão e Desaparecimento do Gradiente

- Saída corresponde à entrada multiplicada pelas matrizes de peso em cada camada
- Com muitas camadas, uma matriz de pesos com valores levemente maiores que a identidade geram valores muito grandes na saída (explosão)
- Se forem um pouco menores que a identidade, vão gerar valores muito pequenos (desaparecimento)
- Mesmo acontece no gradiente
- Problema para o treinamento de redes profundas por muito tempo
- Inicialização adequada ameniza

Explosão e Desaparecimento do Gradiente

- Saída corresponde à entrada multiplicada pelas matrizes de peso em cada camada
- Com muitas camadas, uma matriz de pesos com valores levemente maiores que a identidade geram valores muito grandes na saída (explosão)
- Se forem um pouco menores que a identidade, vão gerar valores muito pequenos (desaparecimento)
- Mesmo acontece no gradiente
- Problema para o treinamento de redes profundas por muito tempo
- Inicialização adequada ameniza

Explosão e Desaparecimento do Gradiente

- Saída corresponde à entrada multiplicada pelas matrizes de peso em cada camada
- Com muitas camadas, uma matriz de pesos com valores levemente maiores que a identidade geram valores muito grandes na saída (explosão)
- Se forem um pouco menores que a identidade, vão gerar valores muito pequenos (desaparecimento)
- Mesmo acontece no gradiente
- Problema para o treinamento de redes profundas por muito tempo
- Inicialização adequada ameniza

Explosão e Desaparecimento do Gradiente

- Saída corresponde à entrada multiplicada pelas matrizes de peso em cada camada
- Com muitas camadas, uma matriz de pesos com valores levemente maiores que a identidade geram valores muito grandes na saída (explosão)
- Se forem um pouco menores que a identidade, vão gerar valores muito pequenos (desaparecimento)
- Mesmo acontece no gradiente
- Problema para o treinamento de redes profundas por muito tempo
- Inicialização adequada ameniza

Explosão e Desaparecimento do Gradiente

- Saída corresponde à entrada multiplicada pelas matrizes de peso em cada camada
- Com muitas camadas, uma matriz de pesos com valores levemente maiores que a identidade geram valores muito grandes na saída (explosão)
- Se forem um pouco menores que a identidade, vão gerar valores muito pequenos (desaparecimento)
- Mesmo acontece no gradiente
- Problema para o treinamento de redes profundas por muito tempo
- Inicialização adequada ameniza

Explosão e Desaparecimento do Gradiente

- Saída corresponde à entrada multiplicada pelas matrizes de peso em cada camada
- Com muitas camadas, uma matriz de pesos com valores levemente maiores que a identidade geram valores muito grandes na saída (explosão)
- Se forem um pouco menores que a identidade, vão gerar valores muito pequenos (desaparecimento)
- Mesmo acontece no gradiente
- Problema para o treinamento de redes profundas por muito tempo
- Inicialização adequada ameniza

Inicialização dos Pesos em Redes Profundas

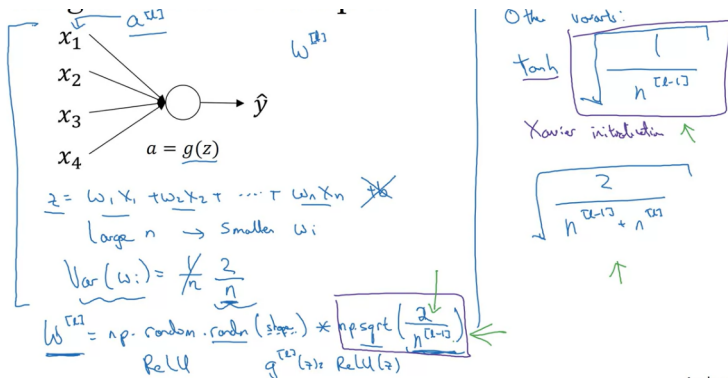
- Quanto maior o número de entradas (n), menor os pesos precisam ser para evitar o desaparecimento/explosão
- Variância dos pesos $1/n$ na sigmoide (ou $2/n$ na ReLU)
- Variável Gaussiana com variância 1 multiplicada por esse fator

Inicialização dos Pesos em Redes Profundas

- Quanto maior o número de entradas (n), menor os pesos precisam ser para evitar o desaparecimento/explosão
- Variância dos pesos $1/n$ na sigmoide (ou $2/n$ na ReLU)
- Variável Gaussiana com variância 1 multiplicada por esse fator

Inicialização dos Pesos em Redes Profundas

- Quanto maior o número de entradas (n), menor os pesos precisam ser para evitar o desaparecimento/explosão
- Variância dos pesos $1/n$ na sigmoide (ou $2/n$ na ReLU)
- Variável Gaussiana com variância 1 multiplicada por esse fator



Inicialização dos Pesos em Redes Profundas

- NOTA: O método que multiplica por $\sqrt{2/n^{[l-1]}}$ é devido a Kaiming He; por $\sqrt{1/n^{[l-1]}}$ é devido a Xavier; por $\sqrt{2/(n^{[l-1]} + n^{[l]})}$ é devido a Yoshua Bengio
- Podemos testar valores de variância como um hiperparâmetro a mais, mas isso raramente vai valer a pena

Inicialização dos Pesos em Redes Profundas

- NOTA: O método que multiplica por $\sqrt{2/n^{[l-1]}}$ é devido a Kaiming He; por $\sqrt{1/n^{[l-1]}}$ é devido a Xavier; por $\sqrt{2/(n^{[l-1]} + n^{[l]})}$ é devido a Yoshua Bengio
- Podemos testar valores de variância como um hiperparâmetro a mais, mas isso raramente vai valer a pena

Checagem do Gradiente

- Identificar *bugs* na implementação do *backpropagation*
- Checagem do cálculo da derivada usando a diferença centrada

$$\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon} \approx f'(\theta)$$

- Erro da diferença centrada é da ordem de ϵ^2 , enquanto para a diferença de um lado é ϵ

Checagem do Gradiente

- Identificar *bugs* na implementação do *backpropagation*
- Checagem do cálculo da derivada usando a diferença centrada

$$\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon} \approx f'(\theta)$$

- Erro da diferença centrada é da ordem de ϵ^2 , enquanto para a diferença de um lado é ϵ

Checagem do Gradiente

- Identificar *bugs* na implementação do *backpropagation*
- Checagem do cálculo da derivada usando a diferença centrada

$$\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon} \approx f'(\theta)$$

- Erro da diferença centrada é da ordem de ϵ^2 , enquanto para a diferença de um lado é ϵ

Checagem do Gradiente - Passos Gerais

- 1 Concatenar parâmetros $W[1], b[1], \dots, W[L], b[L]$ em um vetor gigante θ . Teremos então $J(W[1], b[1], \dots, W[L], b[L]) = J(\theta)$
- 2 Concatenamos os valores dos gradientes $dW[1], db[1], \dots, dW[L], db[L]$ em um vetor gigante $d\theta$. $d\theta$ é então o gradiente de $J(\theta)$
- 3 Para cada i :

$$d\theta_{\text{aprox}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon} \approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$
- 4 Checar $e = \frac{\|d\theta_{\text{aprox}} - d\theta\|_2}{\|d\theta_{\text{aprox}}\|_2 + \|d\theta\|_2}$
- 5 $E \leq 10^{-7} \rightarrow$ Ótimo!; $10^{-3} \leq E < 10^{-7} \rightarrow$ Checar componentes individuais; $E > 10^{-3} \rightarrow$ Problema!

Checagem do Gradiente - Passos Gerais

- 1 Concatenar parâmetros $W[1], b[1], \dots, W[L], b[L]$ em um vetor gigante θ . Teremos então $J(W[1], b[1], \dots, W[L], b[L]) = J(\theta)$
- 2 Concatenamos os valores dos gradientes $dW[1], db[1], \dots, dW[L], db[L]$ em um vetor gigante $d\theta$. $d\theta$ é então o gradiente de $J(\theta)$
- 3 Para cada i :

$$d\theta_{\text{aprox}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon} \approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$
- 4 Checar $e = \frac{\|d\theta_{\text{aprox}} - d\theta\|_2}{\|d\theta_{\text{aprox}}\|_2 + \|d\theta\|_2}$
- 5 $E \leq 10^{-7} \rightarrow$ Ótimo!; $10^{-3} \leq E < 10^{-7} \rightarrow$ Checar componentes individuais; $E > 10^{-3} \rightarrow$ Problema!

Checagem do Gradiente - Passos Gerais

- 1 Concatenar parâmetros $W[1], b[1], \dots, W[L], b[L]$ em um vetor gigante θ . Teremos então $J(W[1], b[1], \dots, W[L], b[L]) = J(\theta)$
- 2 Concatenamos os valores dos gradientes $dW[1], db[1], \dots, dW[L], db[L]$ em um vetor gigante $d\theta$. $d\theta$ é então o gradiente de $J(\theta)$
- 3 Para cada i :

$$d\theta_{\text{aprox}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon} \approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$
- 4 Checar $e = \frac{\|d\theta_{\text{aprox}} - d\theta\|_2}{\|d\theta_{\text{aprox}}\|_2 + \|d\theta\|_2}$
- 5 $E \leq 10^{-7} \rightarrow \text{Ótimo!}; 10^{-3} \leq E < 10^{-7} \rightarrow \text{Checar componentes individuais}; E > 10^{-3} \rightarrow \text{Problema!}$

Checagem do Gradiente - Passos Gerais

- 1 Concatenar parâmetros $W[1], b[1], \dots, W[L], b[L]$ em um vetor gigante θ . Teremos então $J(W[1], b[1], \dots, W[L], b[L]) = J(\theta)$
- 2 Concatenamos os valores dos gradientes $dW[1], db[1], \dots, dW[L], db[L]$ em um vetor gigante $d\theta$. $d\theta$ é então o gradiente de $J(\theta)$
- 3 Para cada i :

$$d\theta_{\text{aprox}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon} \approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$
- 4 Checar $e = \frac{\|d\theta_{\text{aprox}} - d\theta\|_2}{\|d\theta_{\text{aprox}}\|_2 + \|d\theta\|_2}$
- 5 $E \leq 10^{-7} \rightarrow \text{Ótimo!}; 10^{-3} \leq E < 10^{-7} \rightarrow \text{Checar componentes individuais}; E > 10^{-3} \rightarrow \text{Problema!}$

Checagem do Gradiente - Passos Gerais

- 1 Concatenar parâmetros $W[1], b[1], \dots, W[L], b[L]$ em um vetor gigante θ . Teremos então $J(W[1], b[1], \dots, W[L], b[L]) = J(\theta)$
- 2 Concatenamos os valores dos gradientes $dW[1], db[1], \dots, dW[L], db[L]$ em um vetor gigante $d\theta$. $d\theta$ é então o gradiente de $J(\theta)$
- 3 Para cada i :

$$d\theta_{aprox}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon} \approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$
- 4 Checar $e = \frac{\|d\theta_{aprox} - d\theta\|_2}{\|d\theta_{aprox}\|_2 + \|d\theta\|_2}$
- 5 $E \leq 10^{-7} \rightarrow$ Ótimo!; $10^{-3} \leq E < 10^{-7} \rightarrow$ Checar componentes individuais; $E > 10^{-3} \rightarrow$ Problema!

Checagem do Gradiente - Notas de Implementação

- Não use no treino, apenas para debugar; cálculo de $d\theta_{approx}[i]$ é muito caro
- Se falhar, observe cada componente individualmente; problema pode estar, por exemplo, apenas no cálculo de $db[l]$ ou $dW[l]$
- Lembre-se da regularização, derivada $d\theta$ leva o termo regularizador em conta

Checagem do Gradiente - Notas de Implementação

- Não use no treino, apenas para debugar; cálculo de $d\theta_{\text{aprox}}[i]$ é muito caro
- Se falhar, observe cada componente individualmente; problema pode estar, por exemplo, apenas no cálculo de $db[l]$ ou $dW[l]$
- Lembre-se da regularização, derivada $d\theta$ leva o termo regularizador em conta

Checagem do Gradiente - Notas de Implementação

- Não use no treino, apenas para debugar; cálculo de $d\theta_{\text{aprox}}[i]$ é muito caro
- Se falhar, observe cada componente individualmente; problema pode estar, por exemplo, apenas no cálculo de $db[l]$ ou $dW[l]$
- Lembre-se da regularização, derivada $d\theta$ leva o termo regularizador em conta

Checagem do Gradiente - Notas de Implementação

- Não funciona com *dropout*
- Desligar o *dropout* (preferível) no grad check ou aplicar o grad check apenas na parte dos nós que são mantidos
- Embora seja raro, pode acontecer de o *backpropagation* funcionar corretamente, para $W, b \approx 0$ na inicialização, e depois falhar para W, b grandes
- Solução: rodar o grad check na inicialização e depois rodar novamente após algumas épocas

Checagem do Gradiente - Notas de Implementação

- Não funciona com *dropout*
- Desligar o *dropout* (preferível) no grad check ou aplicar o grad check apenas na parte dos nós que são mantidos
- Embora seja raro, pode acontecer de o *backpropagation* funcionar corretamente, para $W, b \approx 0$ na inicialização, e depois falhar para W, b grandes
- Solução: rodar o grad check na inicialização e depois rodar novamente após algumas épocas

Checagem do Gradiente - Notas de Implementação

- Não funciona com *dropout*
- Desligar o *dropout* (preferível) no grad check ou aplicar o grad check apenas na parte dos nós que são mantidos
- Embora seja raro, pode acontecer de o *backpropagation* funcionar corretamente, para $W, b \approx 0$ na inicialização, e depois falhar para W, b grandes
- Solução: rodar o grad check na inicialização e depois rodar novamente após algumas épocas

Checagem do Gradiente - Notas de Implementação

- Não funciona com *dropout*
- Desligar o *dropout* (preferível) no grad check ou aplicar o grad check apenas na parte dos nós que são mantidos
- Embora seja raro, pode acontecer de o *backpropagation* funcionar corretamente, para $W, b \approx 0$ na inicialização, e depois falhar para W, b grandes
- Solução: rodar o grad check na inicialização e depois rodar novamente após algumas épocas